

Estructuras de Datos y Algoritmos

Tema 2: Complejidad algorítmica

Dr. Jorge Armando Ramírez Díaz

Universidad Nacional Autónoma de México
ENES Unidad Juriquilla
Licenciatura en Matemáticas para el Desarrollo

Si vamos a resolver un problema puede que existan varias opciones de algoritmos.

¿Como elegimos entre ellos?

En general hay dos enfoques:

- Empirico, consiste en programar las opciones y probarlas bajo distintos casos.
- Teorico, consiste en determinar matematicamente la cantidad de recursos necesarios (tiempo de computo y espacio de almacenamiento).

Importancia

Dos algoritmos que resuelven el mismo problema pueden escalar diferente en recursos a medida que crecen sus entradas n .

Entrada n	Logarítmica	Lineal	Cuadrática	Exponencial
10	3	10	100	1,024
100	7	100	10,000	$\approx 10^{30}$
1,000	10	1,000	10^6	$\approx 10^{301}$
10,000	14	10,000	10^8	$\approx 10^{3010}$

La tabla compara órdenes de magnitud de operaciones.

¿Qué herramienta nos permite comparar estas formas de crecimiento?

Modelo RAM

Usamos una computadora idealizada para contar operaciones de forma consistente, llamada *Random Access Machine* (RAM). Esto es una abstracción.

Supuesto del modelo RAM	En una máquina real
Operación básica = un paso.	Algunas operaciones cuestan más que otras.
Ciclos y subrutinas se descomponen en operaciones.	Compilador y paralelismo alteran los tiempos.
Acceso a memoria = un paso.	La jerarquía de memoria cambia el costo de acceso.

Notas

- Se mide la cantidad de operaciones en lugar del tiempo ya que el tiempo depende mayormente del hardware.
- Siempre va a interesar el peor caso.
- La complejidad se mide en función del tamaño de la entrada.

Ejemplo Modelo RAM

Considere el algoritmo siguiente:

Algoritmo de ejemplo

Entrada: $n \in \mathbb{Z}_+$

Inicializar $x \leftarrow 0, i \leftarrow 1$

Repetir mientras $i \leq n$:

$x \leftarrow x + 1$

$i \leftarrow i + 1$

Esto se ve en python como

```
def algoritmo_ejemplo(n):  
    x = 0  
    for i = 1 to n:  
        x += 1
```

Modelo RAM

Operación	Veces
$x = 0$	1
$i = 1$	1
$i \leq n$	$n + 1$
$x = x + 1$	$2n$
$i = i + 1$	$2n$

$$T(n) = 1 + 1 + (n + 1) + 2n + 2n = 5n + 3$$

Ejemplo Modelo RAM: suma de un arreglo

Considere el algoritmo siguiente:

Suma de los elementos

Entrada: $A[0], \dots, A[n-1]$, con $n \in \mathbb{Z}_+$

Inicializar $total \leftarrow 0, i \leftarrow 0$

Repetir mientras $i < n$:

$total \leftarrow total + A[i]$

$i \leftarrow i + 1$

Regresar $total$

Esto se ve en Python como:

```
def suma_elementos(A, n):  
    total = 0  
    i = 0  
    while i < n:  
        total = total + A[i]  
        i = i + 1  
    return total
```

Modelo RAM

Operación	Veces
$total = 0$	1
$i = 0$	1
$i < n$	$n + 1$
$total = total + A[i]$	$3n$
$i = i + 1$	$2n$
return $total$	1

En $total = total + A[i]$ contamos un acceso al arreglo, una suma y una asignación por iteración.

$$T(n) = 1 + 1 + (n + 1) + 3n + 2n + 1 = 6n + 4$$

Actividad Modelo RAM: suma de la diagonal

Algoritmo

Entrada: matriz cuadrada A de tamaño $n \times n$
Inicializar $total \leftarrow 0$
Sumar en $total$ cada elemento de la diagonal principal
Regresar $total$

Implementación con ciclos anidados

```
def diagonal_anidada(A):  
    total = 0  
    i = 0  
    n = len(A)  
    while i < n:  
        j = 0  
        while j < n:  
            if i == j:  
                total = total + A[i][j]  
            j = j + 1  
        i = i + 1  
    return total
```

$$T_{\text{anidada}}(n) = \underline{\hspace{2cm}}$$

Considere cada acceso a una celda $A[i][j]$ como una operación. ¿Cuál implementación requiere menos pasos cuando crece n ?

Implementación con un ciclo

```
def diagonal_directa(A):  
    total = 0  
    i = 0  
    n = len(A)  
    while i < n:  
        total = total + A[i][i]  
        i = i + 1  
    return total
```

$$T_{\text{directa}}(n) = \underline{\hspace{2cm}}$$

Dependiendo de las entradas en los algoritmos, puede existir tres casos de complejidad:

- El mejor caso: el minimo numero de pasos que podria tomar el algoritmo
- El peor caso: el maximo numero de pasos que podria tomar el algoritmo. Usualmente el mas adecuado para problemas criticos.
- El caso promedio: el promedio de pasos que podria tomar el algoritmo

Ejemplo de casos de complejidad

Considere el siguiente algoritmo:

Ejemplo

Entrada: $n \in \mathbb{Z}_+$

Inicializar $x \leftarrow U(1, n), i \leftarrow 1$

Repetir mientras $i \neq x$:

$i \leftarrow i + 1$

Esto se ve en python como

```
import numpy as np

def algoritmo_ejemplo(n):
    x = np.random.randint(1, n+1)
    i = 1
    while i != x:
        i += 1
```

Modelo RAM

Operación	Veces
$x = np.random.randint(1, n + 1)$	1
$i = 1$	1
$i \neq x$	x
$i = i + 1$	$2(x - 1)$

$$T(n) = 3x$$

Pero como $x \sim U(1, n)$ podría ocurrir lo siguiente:

$$x_{\text{mejor}} = 1$$

$$x_{\text{peor}} = n$$

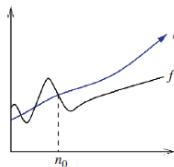
$$x_{\text{promedio}} = \frac{1}{n} \sum_{k=1}^n k = \frac{n+1}{2}$$

Cotas de crecimiento

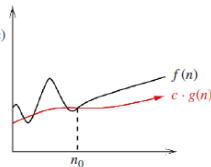
En vez de conservar una función exacta complicada, buscamos funciones simples que la acoten para n grande.

$$c_1 g(n) \leq f(n) \leq c_2 g(n)$$

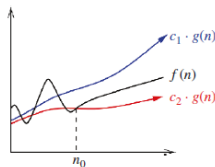
- $c_2 g(n)$: cota superior, $O(g(n))$.
- $c_1 g(n)$: cota inferior, $\Omega(g(n))$.
- Ambas cotas: mismo orden, $\Theta(g(n))$.



(a)



(b)



(c)

Ejemplo

Sea una función y su cota candidata:

$$f(n) = 3n^2 + 20n + 7$$

$$g(n) = n^2$$

Definiciones formales de las cotas asintóticas

Sean $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$, con $g(n) > 0$ para n suficientemente grande.

Definiciones

$$f \in O(g) \iff \exists c > 0, \exists n_0 \in \mathbb{N}, \forall n \geq n_0 : 0 \leq f(n) \leq c g(n).$$

$$f \in \Omega(g) \iff \exists c > 0, \exists n_0 \in \mathbb{N}, \forall n \geq n_0 : 0 \leq c g(n) \leq f(n).$$

$$f \in \Theta(g) \iff \exists c_1, c_2 > 0, \exists n_0 \in \mathbb{N}, \forall n \geq n_0 : \\ 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n).$$

$O(g)$, $\Omega(g)$ y $\Theta(g)$ son conjuntos de funciones.

Teorema: relación entre O , Ω y Θ

Teorema

Para $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$,

$$f \in \Theta(g) \iff f \in O(g) \text{ y } f \in \Omega(g).$$

Equivalentemente, $\Theta(g) = O(g) \cap \Omega(g)$.

Demostración

($\Rightarrow$) Si $f \in \Theta(g)$, existen $c_1, c_2 > 0$ y n_0 tales que, para todo $n \geq n_0$,

$$c_1 g(n) \leq f(n) \leq c_2 g(n).$$

La desigualdad izquierda prueba $f \in \Omega(g)$ y la derecha prueba $f \in O(g)$.

($\Leftarrow$) Si $f \in O(g)$ desde n_O con constante c_2 , y $f \in \Omega(g)$ desde n_Ω con constante c_1 , tomamos $n_0 = \max\{n_O, n_\Omega\}$. Para todo $n \geq n_0$ se cumplen ambas desigualdades; por tanto, $f \in \Theta(g)$. $\square$

Ejemplo de cotas

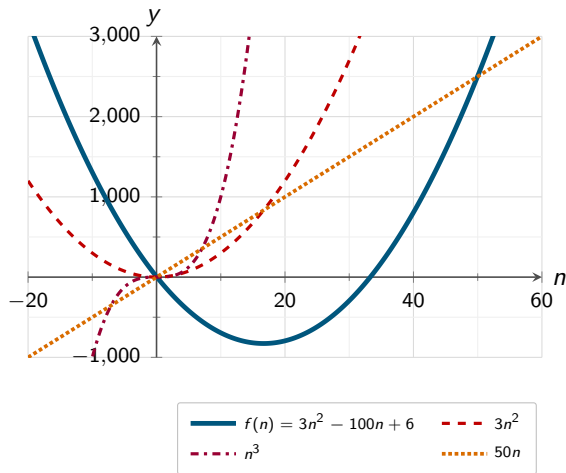
Sea la función

$$f(n) = 3n^2 - 100n + 6.$$

$$c = 3 : f(n) \leq 3n^2, \forall n \geq 1 \implies f \in O(n^2).$$

$$c = 1 : f(n) \leq n^3, \forall n \geq 1 \implies f \in O(n^3).$$

$$\forall c > 0, \exists n_0 : cn < f(n), \forall n \geq n_0 \implies f \notin O(n).$$



Ejemplo de cotas

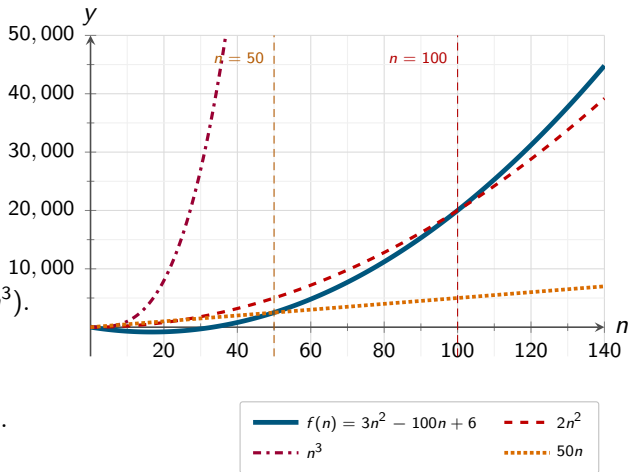
Sea la función

$$f(n) = 3n^2 - 100n + 6.$$

$$c = 2 : 2n^2 \leq f(n), \forall n \geq 100 \implies f \in \Omega(n^2).$$

$$\forall c > 0, \exists n_0 : f(n) < cn^3, \forall n \geq n_0 \implies f \notin \Omega(n^3).$$

$$\forall c > 0, \exists n_0 : cn < f(n), \forall n \geq n_0 \implies f \in \Omega(n).$$



Ejemplo de cotas

Sea la función

$$f(n) = 3n^2 - 100n + 6.$$

Entonces, combinando las cotas anteriores

$$f \in O(n^2) \text{ y } f \in \Omega(n^2) \implies \boxed{f \in \Theta(n^2)}$$

$$f \in O(n^3) \text{ y } f \notin \Omega(n^3) \implies f \notin \Theta(n^3)$$

$$f \notin O(n) \text{ y } f \in \Omega(n) \implies f \notin \Theta(n)$$

Familias de crecimiento

Estas son algunas de las familias de crecimiento que se pueden encontrar como cota

Clase	Nombre	Intuición
$f(n) = c$	Constante	Sin crecimiento
$f(n) = \log n$	Logarítmica	Crece despacio a medida que crece n
$f(n) = n$	Lineal	Crece proporcional a n
$f(n) = n^2$	Cuadrática	Crecimiento rápido
$f(n) = c^n$	Exponencial	Crecimiento muy rápido
$f(n) = n!$	Factorial	Crecimiento extremadamente rápido

$$n! \gg 2^n \gg n^3 \gg n^2 \gg n \log n \gg n \gg \log n \gg 1$$

Ejemplo

Considere el algoritmo siguiente:

```
x = 0
for i = 1 to n:
    x = x + 1
```

RAM

Operación	Veces
$x = 0$	1
$i = 1$	1
$i \leq n$	$n + 1$
$x = x + 1$	$2n$
$i = i + 1$	$2n$

$$T(n) = 1 + 1 + (n + 1) + 2n + 2n = 5n + 3$$

Cotas

$$5n + 3 \leq 8n, \quad n \geq 1$$

$$\Rightarrow T(n) \in O(n)$$

$$5n + 3 \geq 5n, \quad n \geq 1$$

$$\Rightarrow T(n) \in \Omega(n)$$

$$T(n) \in O(n) \cap \Omega(n)$$

$$\Rightarrow \boxed{T(n) \in \Theta(n)}$$

Ejemplo: ciclos consecutivos

Considere el algoritmo siguiente:

```
x = 0
for i = 1 to n:
    x = x + 1
for j = 1 to n:
    x = x + 1
```

RAM

Operación	Veces
$x = 0$	1
$i = 1, j = 1$	2
$i \leq n, j \leq n$	$2(n + 1)$
$x = x + 1$	$4n$
$i = i + 1, j = j + 1$	$4n$

$$T(n) = 1 + 2 + 2(n + 1) + 4n + 4n = 10n + 5$$

Cotas

$$10n + 5 \leq 15n, \quad n \geq 1$$

$$\Rightarrow T(n) \in O(n)$$

$$10n + 5 \geq 10n, \quad n \geq 1$$

$$\Rightarrow T(n) \in \Omega(n)$$

$$T(n) \in O(n) \cap \Omega(n)$$

$$\Rightarrow \boxed{T(n) \in \Theta(n)}$$

Ejemplo: ciclos anidados

Considere el algoritmo siguiente:

```
x = 0
for i = 1 to n:
    for j = 1 to n:
        x = x + 1
```

RAM

Operación	Veces
$x = 0$	1
$i = 1$	1
$i \leq n$	$n + 1$
$i = i + 1$	$2n$
$j = 1$	n
$j \leq n$	$n(n + 1)$
$x = x + 1$	$2n^2$
$j = j + 1$	$2n^2$

$$T(n) = 5n^2 + 5n + 3$$

Cotas

$$5n^2 + 5n + 3 \leq 13n^2, \quad n \geq 1$$
$$\Rightarrow T(n) \in O(n^2)$$

$$5n^2 + 5n + 3 \geq 5n^2, \quad n \geq 1$$
$$\Rightarrow T(n) \in \Omega(n^2)$$

$$T(n) \in O(n^2) \cap \Omega(n^2)$$
$$\Rightarrow \boxed{T(n) \in \Theta(n^2)}$$

Actividad: conteo RAM y cotas

Complete el conteo RAM:

```
x = 0
for i = 1 to n:
  y = 0
  z = 0
  for j = 1 to n:
    y = y + 1
    for k = 1 to n:
      z = z + 1
  x = x + y + z
```

Complete las cotas

$$T(n) \leq \text{_____}, \quad n \geq \text{_____}$$

$$\Rightarrow T(n) \in O(\text{_____})$$

$$T(n) \geq \text{_____}, \quad n \geq \text{_____}$$

$$\Rightarrow T(n) \in \Omega(\text{_____})$$

$$\boxed{T(n) \in \Theta(\text{_____})}$$

Ejemplo logarítmico

Considere el algoritmo:

```
x = 1
while x < n:
    x = 2 * x
```

Idea

Después de k iteraciones,

$$x = 2^k.$$

El ciclo termina cuando $2^k \geq n$.

Traza para $n = 20$

k	0	1	2	3	4	5
x	1	2	4	8	16	32

La condición es verdadera para $x = 1, 2, 4, 8, 16$ y falsa para $x = 32$.

El menor entero k que satisface $2^k \geq n$ es

$$k = \lceil \log_2 n \rceil.$$

Para $n = 20$, el cuerpo se ejecuta 5 veces.

Ejemplo logarítmico

Sea

$$k = \lceil \log_2 n \rceil.$$

Conteo RAM

Operación	Veces
$x = 1$	1
$x < n$	$k + 1$
$x = 2 * x$	$2k$

$$\begin{aligned}T(n) &= 1 + (k + 1) + 2k \\&= 3k + 2 \\&= 3 \lceil \log_2 n \rceil + 2.\end{aligned}$$

Generación de las cotas

Para $n \geq 2$,

$$\log_2 n \leq \lceil \log_2 n \rceil$$

Por tanto,

$$T(n) \geq 3 \log_2 n \Rightarrow T(n) \in \Omega(\log n),$$

y

$$T(n) \leq 8 \log_2 n \Rightarrow T(n) \in O(\log n).$$

$$T(n) \in \Theta(\log n)$$

Ejemplo exponencial

Considere el siguiente algoritmo:

```
limite = 1
for i = 1 to n:
    limite = 2 * limite

x = 0
while x < limite:
    x = x + 1
```

Traza para $n = 3$

i	0	1	2	3
limite	1	2	4	8

Al terminar el primer ciclo,

$$\text{limite} = 2^n = 2^3 = 8.$$

Ejemplo exponencial

Al terminar el primer ciclo, $L = \text{limite} = 2^n$.

Conteo RAM

Operación	Veces
<code>limite = 1</code>	1
<code>i = 1</code>	1
<code>i ≤ n</code>	$n + 1$
<code>limite = 2 * limite</code>	$2n$
<code>i = i + 1</code>	$2n$
<code>x = 0</code>	1
<code>x < limite</code>	$2^n + 1$
<code>x = x + 1</code>	$2 \cdot 2^n$

$$\begin{aligned}T(n) &= 1 + 1 + (n + 1) + 2n + 2n + 1 \\&\quad + (2^n + 1) + 2 \cdot 2^n \\&= 3 \cdot 2^n + 5n + 5.\end{aligned}$$

Generación de las cotas

Para $n \geq 1$, se cumple $n \leq 2^n$ y $1 \leq 2^n$.

$$T(n) \geq 3 \cdot 2^n \Rightarrow T(n) \in \Omega(2^n)$$

$$T(n) \leq 13 \cdot 2^n \Rightarrow T(n) \in O(2^n)$$

por lo que

$T(n) \in \Theta(2^n)$

Ejemplo factorial

Considere el siguiente algoritmo:

```
limite = 1
for i = 1 to n:
    limite = limite * i

x = 0
while x < limite:
    x = x + 1
```

El primer ciclo calcula $n!$; el segundo realiza esa cantidad de iteraciones.

Traza para $n = 4$

i	inicial	1	2	3	4
limite	1	1	2	6	24

Al terminar el primer ciclo,

$$\text{limite} = 1 \cdot 2 \cdot 3 \cdot 4 = 4! = 24.$$

Ejemplo factorial

Al terminar el primer ciclo, $L = \text{limite} = n!$.

Conteo RAM

Operación	Veces
<code>limite = 1</code>	1
<code>i = 1</code>	1
<code>i ≤ n</code>	$n + 1$
<code>limite = limite * i</code>	$2n$
<code>i = i + 1</code>	$2n$
<code>x = 0</code>	1
<code>x < limite</code>	$n! + 1$
<code>x = x + 1</code>	$2n!$

$$\begin{aligned}T(n) &= 1 + 1 + (n + 1) + 2n + 2n + 1 \\&\quad + (n! + 1) + 2n! \\&= 3n! + 5n + 5.\end{aligned}$$

Generación de las cotas

Para $n \geq 1$, se cumple $n \leq n!$ y $1 \leq n!$.

$$T(n) \geq 3n! \Rightarrow T(n) \in \Omega(n!).$$

$$T(n) \leq 13n! \Rightarrow T(n) \in O(n!)$$

por lo que

$T(n) \in \Theta(n!)$

No confundir

El primer ciclo calcula $n!$ en tiempo lineal. El crecimiento factorial aparece porque el segundo ejecuta $n!$ iteraciones.

Simplificacion en el analisis

Suma

La suma de dos funciones sigue a la funcion dominante

$$f(n) + g(n) \rightarrow \Theta(\max(f(n), g(n)))$$

Esto se debe a que la funcion mas grande va a proveer el valor mas grande conforme $n \rightarrow \infty$.

Multiplicacion

La mutiplicacion por cualquier constante no afecta su comportamiento asintotico, siempre que $c > 0$

$$\Theta(c \cdot f(n)) \rightarrow \Theta(f(n))$$

Cuando tenemos el caso de dos funciones en un producto, ambos son importantes

$$\Theta(f(n)) \cdot \Theta(g(n)) \rightarrow \Theta(f(n) \cdot g(n))$$

Ejemplo

Considere el algoritmo siguiente:

```
x = 0
for i = 1 to n:
    x = x + 1
```

¿Y si solo analizamos el algoritmo desde dentro hacia afuera?

- 1 Hay una asignacion y una suma dentro del ciclo for, es decir que $O(2)$
- 2 El ciclo for se ejecuta n veces, es decir que $O(n)$
- 3 Hay una asignacion inicial, es decir $O(1)$
- 4 El orden del ciclo for permanece frente a los demas

Pausa matemática: distancias unitarias

Problema

Entre n puntos del plano, $u(n)$ es el máximo número de pares separados exactamente por una unidad.

Lo que Erdős esperaba Conjeturó que, para cualquier constante $\varepsilon > 0$,

$$u(n) \in O(n^{1+\varepsilon}).$$

La mejor cota superior conocida

$$u(n) \in O(n^{4/3})$$

En mayo del 2026 se probó

$$\begin{array}{l} \exists \delta > 0 \text{ fijo :} \\ u(n) \in \Omega(n^{1+\delta}) \text{ para infinitos } n. \end{array}$$

¿Por qué refuta la conjetura?

La conjetura debía cumplirse para cualquier $\varepsilon > 0$. Como δ es fijo, podemos elegir $\varepsilon < \delta$. Entonces $n^{1+\delta}$ crece más rápido que $n^{1+\varepsilon}$, contradiciendo la cota propuesta por Erdős. El orden exacto $\Theta(u(n))$ aún se desconoce. Tampoco conocemos una fórmula general para $u(n)$.

Ejemplo: secuencia de Fibonacci

Idea

La secuencia de Fibonacci es una sucesión en la que cada término, a partir del tercero, se obtiene sumando los dos términos anteriores.

Definición formal

Para $n \in \mathbb{N}$, usamos la convención

$$F_i = \begin{cases} 0, & i = 0, \\ 1, & i = 1, \\ F_{i-1} + F_{i-2}, & i \geq 2. \end{cases}$$

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

Fibonacci: forma recursiva e iterativa

Algoritmo FibonacciRecursivo(i)

Entrada: entero $i \geq 0$

Salida: F_i

Si $i \leq 1$, regresar $F_i \leftarrow i$

En otro caso, calcular $a \leftarrow \text{FibonacciRecursivo}(i - 1)$

Calcular $b \leftarrow \text{FibonacciRecursivo}(i - 2)$

Regresar $F_i \leftarrow a + b$

Algoritmo FibonacciIterativo(i)

Entrada: entero $i \geq 0$

Salida: F_i

Si $i \leq 1$, regresar $F_i \leftarrow i$

Inicializar $\text{anterior} \leftarrow 0$, $\text{actual} \leftarrow 1$

Para $j \leftarrow 2$ hasta i :

$\text{siguiente} \leftarrow \text{anterior} + \text{actual}$

Actualizar $\text{anterior} \leftarrow \text{actual}$

Actualizar $\text{actual} \leftarrow \text{siguiente}$

Regresar $F_i \leftarrow \text{actual}$